

Programació per a *Data Science*

Unitat 7: Anàlisi de dades en Python

En aquest Notebook trobareu un conjunt d'exercicis per practicar. Aquests exercicis no puntuen per la PAC, però us recomanem que els intenteu resoldre com a part del procés d'aprenentatge. Trobareu exemples de possibles solucions als exercicis en el propi notebook, però és important que intenteu resoldre'ls vosaltres abans de consultar les solucions. Les solucions us permetran validar les vostres respostes, així com veure alternatives de resolució de les activitats. També us animem a preguntar qualsevol dubte que sorgeixi sobre la resolució de les activitats per a practicar en el fòrum de l'aula.

Preguntes i exercicis per practicar

Pregunta 1

Investiga i descriu el funcionament de l'mètode de cross-validation *Leave One Out*. Posa un exemple d'aplicació de l'algorisme per resoldre un problema d'anàlisi en l'àmbit de la salut. Recordeu que cal citar les referències consultades per respondre la pregunta, i que la resposta que proporcioneu ha de ser original (redactada per vosaltres mateixos, després d'haver llegit i entès les referències que considereu oportunes).

Resposta

Pregunta 2

Hem d'evitar avaluar els models amb les mateixes dades que s'han utilitzat per a l'aprenentatge. De no fer-ho, podríem afavorir el problema del **sobre-ajust**. En què consisteix aquest problema i quines conseqüències pot tenir?

Resposta

Exercici 1

Carregueu el conjunt de dades Iris incorporat a la llibreria `sklearn`. Implementeu una funció, `describe_iris`, que retorni un diccionari amb la següent estructura:

```
{
    "categories": [],
    "atributs": [],
    "num_mostres": 0
}
```

Categories ha de ser una matriu amb el nom dels **targets** del *dataset*. *Atributs* ha de ser una matriu amb el nom dels **atributs** i finalment, *num_mostres* ha d'indicar el nombre **total de mostres** de l'*dataset*.

In [1]:

```
# Resposta
```

Exercici 2

Representeu gràficament en un *scatter plot* la longitud dels sèpals en funció de l'ample dels sèpals. Nota: per poder incloure accents en els textos de les etiquetes o del títol del plot, cal indicar explícitament que les cadenes de caràcters són unicode. Podeu fer-ho incloent una *u* davant de les cometes que delimiten la cadena de caràcters.

In [2]:

```
# Resposta
```

Exercici 3

Dividir les dades Iris en dos subconjunts, dades d'entrenament i test, en una proporció 70% entrenament i 30% test.

In [3]:

```
# Resposta
```

Exercici 4

Apliqueu el classificador *KNeighborsClassifier* per predir el tipus d'espècie d'iris utilitzant la longitud i ample dels pètals com a atributs i utilitzant 60% de les mostres per entrenament i 40% de les mostres per al test (podeu fer servir qualsevol partició de mostres d'entrenament i de test).

Quin rendiment s'obté?

In [4]:

```
#Resposta
```

Exercici 5

Apliqueu l'algorisme de *clustering KMeans* tal com hem vist al Notebook de teoria, però aquesta vegada utilitzant els següents paràmetres:

```
Número de clústers: 10
Mètode d'inicialització dels punts centrals: 'random'
Nombre d'iteracions per a la selecció de punts centrals: 5
Algoritme: 'elkan'
```

Visualitzeu gràficament el resultat. Si les dades fossin [disperses](#), quina opció del paràmetre *alg* de la funció *sklearn.cluster.KMeans* hauríem d'utilitzar?

In [5]:

```
#Resposta
```

Exercici 6

En els següents exercicis farem servir el conjunt de dades del dataset `breast_cancer` de *sklearn*. Crea una taula amb els estadístics descriptius de totes les característiques dels tumors de la base de dades `breast_cancer` agrupats per tipus (benigne/maligne).

Feu un test estadístic [ttest](#) per comparar l'error de la simetria (*symmetry error*) i finor (*mean smoothness*) dels tumors benignes amb les dels malignes. Donat el resultat obtingut, indica quines característiques podrien ser més rellevants per diagnosticar un tumor maligne.

Nota: Consulta aquest [enllaç](#) si vols ampliar coneixements sobre el test estadístic ttest. Per fer el test estadístic automàticament podeu utilitzar el mètode `ttest_ind` de *Scipy.stats*

In [6]:

```
# Resposta
```

Resposta

Exercici 7

Representa mitjançant [boxplots](#) com varien la textura, l'àrea, la concavitat i la simetria dels tumors segons el tipus (*maligne/benigne*). Recorda ajustar els paràmetres de la visualització per facilitar la lectura i la interpretació de la gràfica.

Nota: Crea un dataframe amb les dades de `diagnostics.data` i `diagnostics.target` per dibuixar els boxplots.

In [7]:

```
# Importem les llibreries

import numpy as np
import pandas as pd
import seaborn as sns # llibreria opcional
import matplotlib.pyplot as plt
from sklearn import datasets

# Importem el dataset
diagnostics = datasets.load_breast_cancer()
```

In [8]:

```
## Resposta
```

Exercici 8

L'objectiu d'un model de **regressió lineal** és trobar una relació entre una o més característiques (variables independents) i una variable objectiu contínua (variable dependent). Quan només fem servir una característica predictiva se l'anomena **Regressió Lineal univariada** i si hi ha múltiples predictors es diu **Regressió Lineal Múltiple**.

Quan la variable dependent és una variable binària que conté dades codificades com 1 (sí, èxit, etc.) o 0 (no, fallada, etc.), com és el cas de l'dataset `breast_cancer`, haurem de fer servir una [regressió logística](#).

Crea un model de regressió logística que estimi la probabilitat que un tumor sigui maligne donada la seva àrea i la seva textura. Mostra la [matriu de confusió](#) del model obtingut.

Nota: Potser t'interessa explorar la funció [sklearn.metrics.confusion_matrix\(\)](#)

In [9]:

```
# Resposta
```

Exercici 9

El **Multi-layer Perceptron (MLP)** és un algorisme d'aprenentatge supervisat que aprèn una funció mitjançant l'entrenament d'un model que associa n dimensions d'entrada (*predictors*) amb dimensions de sortida (*targets*). Tenint en compte un conjunt de funcions i un objectiu, pot aprendre un aproximador de funcions no lineals per classificació o regressió. És diferent de la regressió logística, ja que entre la capa d'entrada i la de sortida, pot haver-hi una o més capes no lineals, anomenades capes ocultes (*hidden layers*).

Aplica un classificador basat en un MLP per predir el tipus de tumor utilitzant l'àrea, la textura, la simetria i la concavitat, com a atributs i utilitzant el 70% de les mostres d'entrenament i el 30% de test. Heu d'utilitzar la funció [sklearn.neural_network.MLPClassifier](#).

Utilitza tres nivells (*layers*) amb 30 neurones per nivell. Quin valor de precisió obtenim en un model basat en un MLP?

In [10]:

```
# Resposta
```

Exercici 10

Apliqueu un classificador basat en un [arbre de decisió](#) d'un màxim de 3 nivells de profunditat per predir els tipus de tumor utilitzant l'àrea, la textura i la simetria com a atributs i utilitzant 60% de les mostres d'entrenament i el 40% de test. Heu d'utilitzar la

funció [sklearn.tree.DecisionTreeClassifier](#).

Quin valor de precisió obtenim en un model basat en un arbre de decisió? Representa l'arbre de decisió i exporta-ho a un arxiu PDF. Explora l'arbre resultant al PDF i respon a la següent pregunta: Donades aquestes dades, com diagnosticaries a un pacient que presenta un tumor d'àrea 400, simetria 0.18, i un valor de textura de 19 punts?

Nota: Potser la funció [tree](#) de *sklearn* sigui d'utilitat.

In [11]:

```
# Resposta
```

Resposta

Solucions exercicis per practicar

Pregunta 1

Investiga i descriu el funcionament de l'mètode de cross-validation *Leave One Out*. Posa un exemple d'aplicació de l'algorisme per resoldre un problema d'anàlisi en l'àmbit de la salut. Recordeu que cal citar les referències consultades per respondre la pregunta, i que la resposta que proporcioneu ha de ser original (redactada per vosaltres mateixos, després d'haver llegit i entès les referències que considereu oportunes).

Resposta

La validació creuada *Leave One Out* és essencialment una estimació del rendiment de generalització d'un model entrenat amb $n - 1$ mostres de dades, que generalment és una estimació lleugerament pessimista del rendiment d'un model entrenat en n mostres. És millor pensar en la validació creuada com una forma d'estimar el rendiment de generalització dels models generats per un procediment particular, en lloc del model en si. Per exemple, imaginem que estem creant un model predictiu del tipus de melanoma (alt o baix risc) i tenim una mostra relativament petita (1000 observacions) que inclouen 3 paràmetres descriptius del melanoma i el seu nivell de risc (alt o baix). Podem ajustar el model usant 999 de les 1000 observacions per després avaluar el seu rendiment donada l'observació que hem deixat fora del set d'entrenament. En aquest cas, podem repetir aquest mètode fins a 999 vegades, per estimar un rendiment mitjà de el model.

Pregunta 2

Hem d'evitar avaluar els models amb les mateixes dades que s'han utilitzat per a l'aprenentatge. De no fer-ho, podríem afavorir el problema del **sobre-ajust**. En què consisteix aquest problema i quines conseqüències pot tenir?

Resposta

L'excés d'ajust d'un model és un error de modelatge que es produeix quan una funció s'ajusta massa a un conjunt limitat de punts de dades. Per tant, només suposa un problema en els models d'aprenentatge supervisat. Una de les conseqüències de el sobre-ajust és que el model pren una forma massa complexa per explicar la idiosincràsia a les dades objecte d'estudi i difícilment generalitza a noves mostres de dades. La **generalització** és un terme utilitzat per descriure la capacitat del model de reaccionar davant les noves dades. És a dir, després de ser entrenat en un entrenament, un model pot digerir dades noves i fer prediccions precises. La capacitat del model per generalitzar és essencial per a l'èxit d'un model. Si un model s'ha format massa bé sobre dades d'entrenament, no podrà generalitzar-se. Farà prediccions inexactes quan es donin noves dades, i farà que el model sigui inútil encara que sigui capaç de fer prediccions precises per a les dades d'entrenament. D'això es diu sobre-ajust (*overfitting*).

Un dels mètodes més utilitzats per avaluar el problema del sobre-ajust es diu **cross-validation**. És un model de validació de models per avaluar com es generalitzen els resultats a un conjunt de dades independent. S'utilitza principalment quan l'objectiu és la predicció i es vol estimar amb exactitud com es comportarà un model predictiu a la pràctica. Quant fem **cross-validation**, destinarem una part de les dades a fer l'entrenament de el model i l'altra part la reservarem per avaluar el rendiment del model après.

Exercici 1

Carregueu el conjunt de dades Iris incorporat a la llibreria `sklearn`. Implementeu una funció, `describe_iris`, que retorni un diccionari amb la següent estructura:

```

    "categories": [],
    "atributs": [],
    "num_mostres": 0
}

```

Categories ha de ser una matriu amb el nom dels **targets** del *dataset*. *Atributs* ha de ser una matriu amb el nom dels **atributs** i finalment, *num_mostres* ha d'indicar el nombre **total de mostres** de l'*dataset*.

In [12]:

```

from sklearn import datasets

#Carreguem el dataset de l'iris

iris = datasets.load_iris()

def describe_iris():
    #Inicialitzem el diccionari
    diccionari = {}
    #Assignem a cada atribut el valor corresponent basant-se el dataset Iris
    diccionari["categorias"] = iris.target_names
    diccionari["atributos"] = iris.feature_names
    diccionari["num_muestras"] = len(iris.data)

    return diccionari

print (describe_iris())

```

```

{'categorias': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'), 'atributos': ['sepal
length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'], 'num_muestras': 150}

```

Exercici 2

Representeu gràficament en un *scatter plot* la longitud dels sèpals en funció de l'ample dels sèpals. Nota: per poder incloure accents en els textos de les etiquetes o del títol del plot, cal indicar explícitament que les cadenes de caràcters són unicode. Podeu fer-ho incloent una *u* davant de les cometes que delimiten la cadena de caràcters.

In [13]:

```

%matplotlib inline

#Importem les llibreries

import matplotlib.pyplot as plt
from sklearn import datasets

# Importem el dataset

iris=datasets.load_iris()

L_sep = iris.data[:,0]

W_sep= iris.data[:,1]

Y = iris.target

# Creem la figura

plt.figure(1, figsize=(8,6))

plt.clf()

plt.scatter(L_sep,W_sep,c=Y)
plt.xlabel(u'Longitud Sépalo')
plt.ylabel(u'Ancho Sépalo')

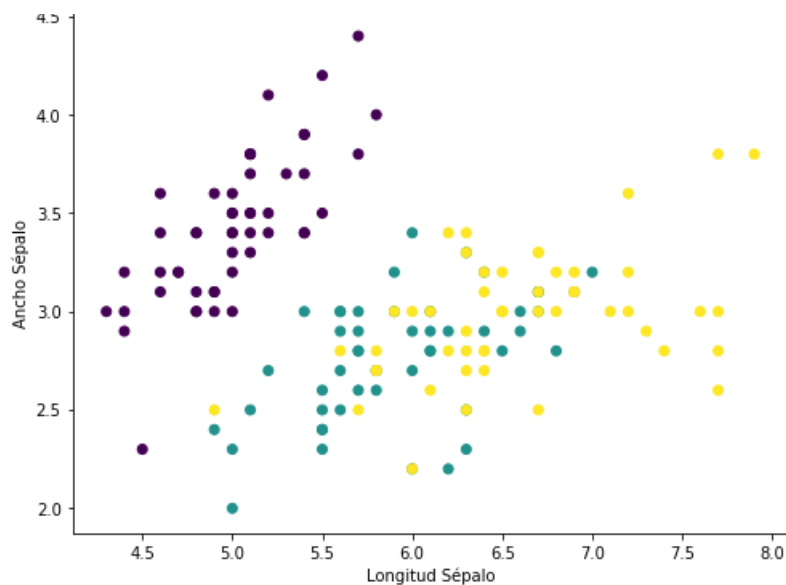
```

Out[13]:

```

Text(0, 0.5, 'Ancho Sépalo')

```



Exercici 3

Dividir les dades Iris en dos subconjunts, dades d'entrenament i test, en una proporció 70% entrenament i 30% test.

In [14]:

```
from sklearn.model_selection import train_test_split

iris=datasets.load_iris()

# Dividim les dades en dades d'entrenament (70%) i de test (30%) mitjançant la funció
train_test_split

train,test = train_test_split(iris.data, test_size=0.3)

# L'atribut test_size = 0.3 divideix les dades en una proporció de 70% i 30%. entrenament = 70% i
test = 30%.

print(train.shape)

print(test.shape)

print(test)
```

```
(105, 4)
(45, 4)
[[5.1 3.7 1.5 0.4]
 [4.9 3.1 1.5 0.2]
 [5.3 3.7 1.5 0.2]
 [4.9 2.4 3.3 1. ]
 [5.1 3.5 1.4 0.3]
 [7.7 2.8 6.7 2. ]
 [5.4 3. 4.5 1.5]
 [5.7 2.9 4.2 1.3]
 [5.5 2.4 3.7 1. ]
 [4.9 2.5 4.5 1.7]
 [5.2 2.7 3.9 1.4]
 [5.4 3.9 1.7 0.4]
 [4.4 3. 1.3 0.2]
 [4.9 3. 1.4 0.2]
 [7.2 3.2 6. 1.8]
 [6.6 3. 4.4 1.4]
 [6.1 2.8 4.7 1.2]
 [5. 2. 3.5 1. ]
 [4.3 3. 1.1 0.1]
 [7.4 2.8 6.1 1.9]
 [6.4 3.2 4.5 1.5]
 [6. 2.2 5. 1.5]
 [7.7 3.8 6.7 2.2]
 [6. 2.9 4.5 1.5]]
```

```
[5.4 3.4 1.7 0.2]
[7.2 3.6 6.1 2.5]
[5.8 2.6 4. 1.2]
[4.6 3.4 1.4 0.3]
[4.4 2.9 1.4 0.2]
[7.3 2.9 6.3 1.8]
[5.5 3.5 1.3 0.2]
[5.9 3. 5.1 1.8]
[6.5 3.2 5.1 2. ]
[7.2 3. 5.8 1.6]
[5. 3.4 1.5 0.2]
[6.3 3.3 4.7 1.6]
[6.6 2.9 4.6 1.3]
[5.7 2.8 4.1 1.3]
[6.4 2.9 4.3 1.3]
[6.9 3.1 4.9 1.5]
[4.8 3.1 1.6 0.2]
[6.2 3.4 5.4 2.3]
[5.7 4.4 1.5 0.4]
[5.8 2.7 5.1 1.9]
[5.8 2.7 4.1 1. ]]
```

Exercici 4

Apliqueu el classificador *KNeighborsClassifier* per predir el tipus d'espècie d'iris utilitzant la longitud i ample dels pètals com a atributs i utilitzant 60% de les mostres per entrenament i 40% de les mostres per al test (podeu fer servir qualsevol partició de mostres d'entrenament i de test).

Quin rendiment s'obté?

In [15]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets

#Carreguen les dades

iris = datasets.load_iris()

iris_df = pd.DataFrame(iris.data)

# Assignem categoria a cadascuna de les espècies
iris_df['Species'] = iris.target
iris_df.loc[(iris_df.Species == 0), "Species"] = iris.target_names[0]
iris_df.loc[(iris_df.Species == 1), "Species"] = iris.target_names[1]
iris_df.loc[(iris_df.Species == 2), "Species"] = iris.target_names[2]

iris_df.columns = ['SepalLength', 'SepalWitdh', 'PetalLength', 'PetalWitdh', 'Species']
```

In [16]:

```
# Resposta

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import warnings
warnings.filterwarnings('ignore')

# Dividim les dades en dades d'entrenament (60%) i de test (40%) mitjançant la funció
train_test_split

train,test = train_test_split(iris_df, test_size=0.4)

# Creem el classificador

knn = KNeighborsClassifier()

# Entrenem el classificador
knn.fit( train[['PetalLength', 'PetalWitdh']],train[['Species']])

# Provem el classificador
```

```
iris_predicted=knn.predict(test[['PetalLength','PetalWitdh']])

# Mostrem els resultats de la predicció sobre el conjunt de test

print ("Accuracy: \t\t" + str(knn.score(test[['PetalLength','PetalWitdh']],test[['Species']]])))
```

Accuracy: 0.9666666666666667

Exercici 5

Apliqueu l'algorisme de *clustering KMeans* tal com hem vist al Notebook de teoria, però aquesta vegada utilitzant els següents paràmetres:

```
Número de clústers: 10
Mètode d'inicialització dels punts centrals: 'random'
Nombre d'iteracions per a la selecció de punts centrals: 5
Algoritme: 'elkan'
```

Visualitzeu gràficament el resultat. Si les dades fossin [disperses](#), quina opció del paràmetre *alg* de la funció *sklearn.cluster.KMeans* hauríem d'utilitzar?

In [17]:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import cluster, datasets

iris = datasets.load_iris()
X_iris = iris.data

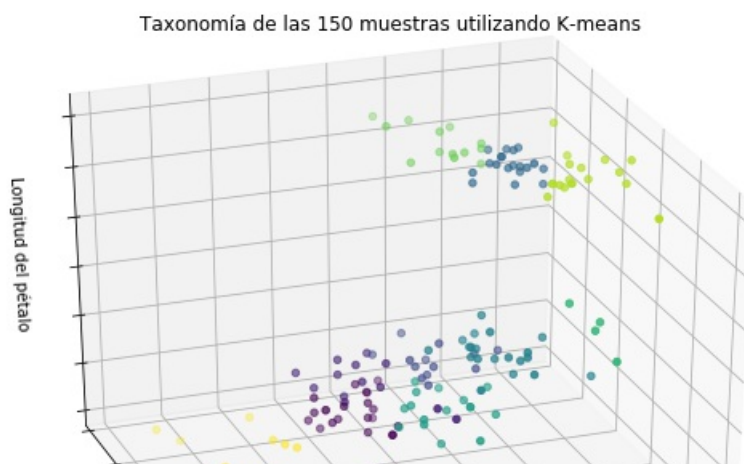
# Consultem l'ajuda de K-means per veure com es defineixen els paràmetres
?cluster.KMeans

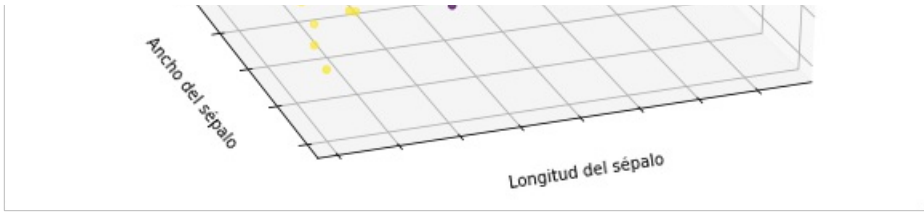
# Carreguem l'algoritme K-means i ajustem els paràmetres segons l'enunciat
k_means = cluster.KMeans(n_clusters=10,init='random',n_init=5, algorithm='elkan')
k_means.fit(X_iris)

# Definim els paràmetres de la figura
fig = plt.figure(1, figsize=(8, 6))
ax = Axes3D(fig, elev=-150, azim=110)
ax.scatter(X_iris[:,[0]], X_iris[:,[1]], X_iris[:,[2]], c=k_means.labels_)
ax.set_title(u"Taxonomía de las 150 muestras utilizando K-means")
ax.set_xlabel(u"Longitud del sépalo")
ax.w_xaxis.set_ticklabels([])
ax.set_ylabel(u"Ancho del sépalo")
ax.w_yaxis.set_ticklabels([])
ax.set_zlabel(u"Longitud del pétalo")
ax.w_zaxis.set_ticklabels([])
```

Out[17]:

[]





Resposta

Si les dades fossin disperses, el paràmetre *algorithm* hauria de ser 'full'.

Exercici 6

En els següents exercicis farem servir el conjunt de dades del dataset `breast_cancer` de *sklearn*. Crea una taula amb els estadístics descriptius de totes les característiques dels tumors de la base de dades `breast_cancer` agrupats per tipus (benigne/maligne).

Feu un test estadístic `ttest` per comparar l'error de la simetria (*symmetry error*) i finor (*mean smoothness*) dels tumors benignes amb les dels malignes. Donat el resultat obtingut, indica quines característiques podrien ser més rellevants per diagnosticar un tumor maligne.

Nota: Consulta aquest [enllaç](#) si vols ampliar coneixements sobre el test estadístic `ttest`. Per fer el test estadístic automàticament podeu utilitzar el mètode `ttest_ind` de *Scipy.stats*

In [18]:

```
# Resposta

# Importem les llibreries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
import scipy.stats as stats

# Importem el dataset
diagnostics = datasets.load_breast_cancer()

# Els transformem en un dataframe
diagnostics_df = pd.DataFrame(diagnostics.data)

# Nombrem les columnes de las variables
diagnostics_df.columns = diagnostics['feature_names']

# Afegim la columna Type amb el resultat de la diagnosi (benigne / maligne)
diagnostics_df['Type'] = diagnostics.target

# Mostrem per pantalla els paràmetres descriptius de la mostra
diagnostics_df.groupby('Type').describe()
```

Out[18]:

	mean radius								mean texture		...	worst symmetry		worst fractal dimension		
	count	mean	std	min	25%	50%	75%	max	count	mean		75%	max	count	mean	std
Type																
0	212.0	17.462830	3.203971	10.950	15.075	17.325	19.59	28.11	212.0	21.604906	...	0.359225	0.6638	212.0	0.091530	0.021
1	357.0	12.146524	1.780512	6.981	11.080	12.200	13.37	17.85	357.0	17.914762	...	0.298300	0.4228	357.0	0.079442	0.016

2 rows x 240 columns

In [19]:

```
# Realitzem un test estadístic per a cada variable comparant els tumors benignes (tipus 0) i malignes (tipus 1).
```

```
# Executem 1 prova t de student per a mostres independents ja que cada registre correspon amb un pacient diferent.
t1, p1 = stats.ttest_ind(diagnostics_df.query("Type==0")['mean smoothness'],diagnostics_df.query("Type==1")['mean smoothness'])
print("Análisis de la tersura. Estadístico t= " + str(t1) + ", p-val = " + str(p1) )
t2, p2 = stats.ttest_ind(diagnostics_df.query("Type==0")['symmetry error'],diagnostics_df.query("Type==1")['symmetry error'])
print("Análisis del error de la simetría. Estadístico t= " + str(t2) + ", p-val = " + str(p2) )
```

Análisis de la tersura. Estadístico t= 9.146098808149038, p-val = 1.0518503592032013e-18

Análisis del error de la simetría. Estadístico t= -0.1552978000059369, p-val = 0.8766418183858812

Resposta

Preferirem la variable finor com diferenciadora dels tumors benignes i malignes ja que el test suggereix que les dues mostres de llisor comparades provenen de poblacions diferents.

Exercici 7

Representa mitjançant [boxplots](#) com varien la textura, l'àrea, la concavitat i la simetria dels tumors segons el tipus (*maligne/benigne*). Recorda ajustar els paràmetres de la visualització per facilitar la lectura i la interpretació de la gràfica.

Nota: Crea un dataframe amb les dades de `diagnostics.data` i `diagnostics.target` per dibuixar els boxplots.

In [20]:

```
# Importem les llibreries

import numpy as np
import pandas as pd
import seaborn as sns # llibreria opcional
import matplotlib.pyplot as plt
from sklearn import datasets

# Importem el dataset
diagnostics = datasets.load_breast_cancer()
```

In [21]:

```
## Resposta
diagnostics_df = pd.DataFrame(diagnostics.data)

# Assignem una categoria a cada tumor
diagnostics_df.columns = diagnostics['feature_names']
diagnostics_df['Type'] = diagnostics.target
diagnostics_df.loc[(diagnostics_df.Type == 0), "Type"] = diagnostics.target_names[0]
diagnostics_df.loc[(diagnostics_df.Type == 1), "Type"] = diagnostics.target_names[1]

# Definim paràmetres de la figura
label=['maligno', 'benigno']
pos=[1,2]
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(15, 10))

# Dibuixem cadascun dels gràfics
vplot1=axes[0,0].boxplot([np.array(diagnostics_df['mean texture'])[diagnostics_df.Type == 'malignant']),
                        np.array(diagnostics_df['mean texture'])[diagnostics_df.Type == 'benign']],vert=False)

axes[0,0].set_xlabel('texture')
axes[0,0].set_ylabel('Type')
axes[0,0].set_yticks(pos)
axes[0,0].set_yticklabels(label)

vplot2=axes[0,1].boxplot([np.array(diagnostics_df['mean area'])[diagnostics_df.Type == 'malignant']),
                        np.array(diagnostics_df['mean area'])[diagnostics_df.Type == 'benign']],vert=False)

axes[0,1].set_xlabel('area')
axes[0,1].set_ylabel('Type')
axes[0,1].set_yticks(pos)
axes[0,1].set_yticklabels(label)
```

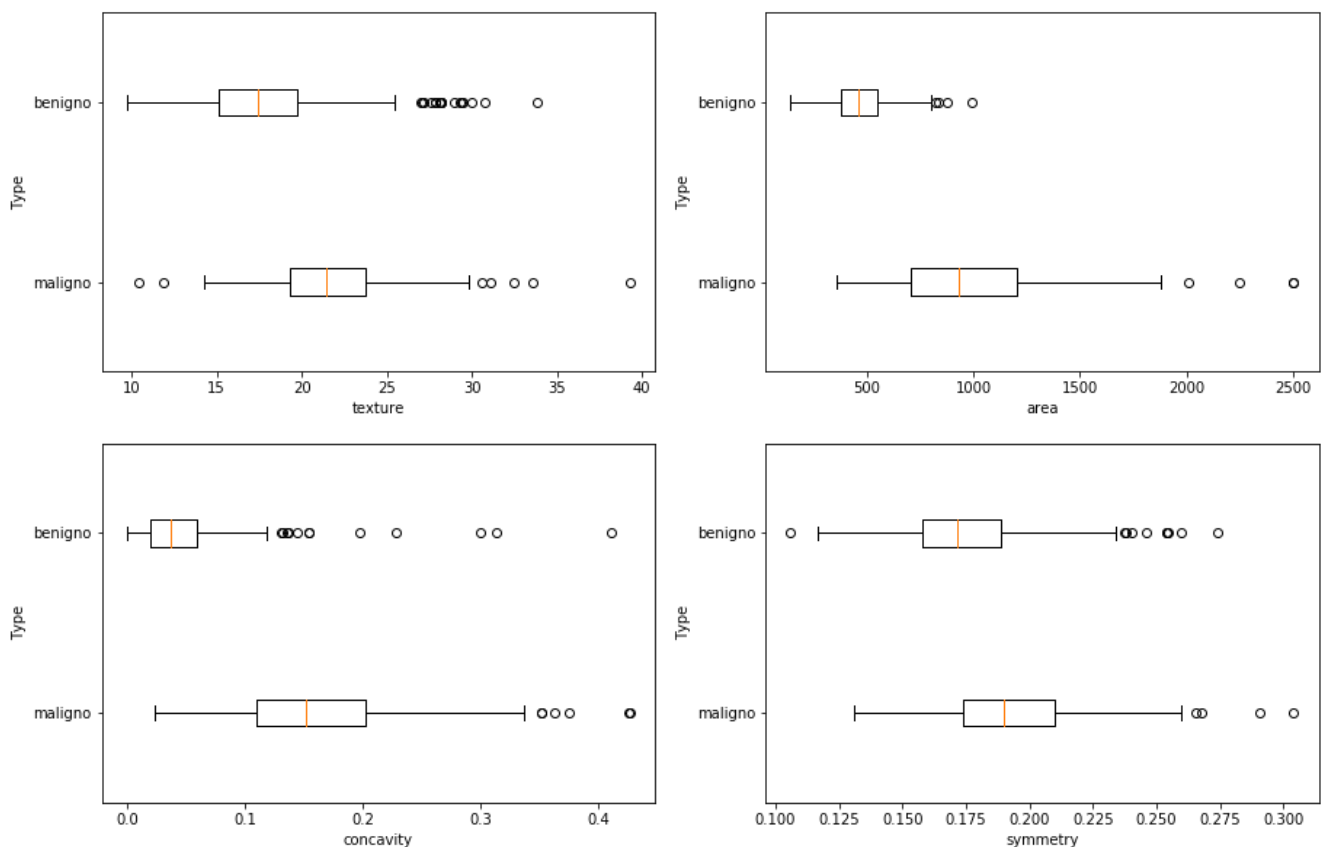
```

vplot3=axes[1,0].boxplot([np.array(diagnostics_df['mean concavity'])[diagnostics_df.Type == 'malignant']),
                          np.array(diagnostics_df['mean concavity'])[diagnostics_df.Type == 'benign']
                          ),vert=False)
axes[1,0].set_xlabel('concavity')
axes[1,0].set_ylabel('Type')
axes[1,0].set_yticks(pos)
axes[1,0].set_yticklabels(label)

vplot4=axes[1,1].boxplot([np.array(diagnostics_df['mean symmetry'])[diagnostics_df.Type == 'malignant']),
                          np.array(diagnostics_df['mean symmetry'])[diagnostics_df.Type == 'benign']
                          ),vert=False)
axes[1,1].set_xlabel('symmetry')
axes[1,1].set_ylabel('Type')
axes[1,1].set_yticks(pos)
axes[1,1].set_yticklabels(label)
colors = ['forestgreen', 'darkorange']

# Assignem color a cada un dels cossos del boxplot
plt.show()

```



In [22]:

```

# Opció 2: Una altra forma de fer l'exercici és mitjançant la llibreria
# Seaborn, que veurem en la propera unitat

# Importem les llibreries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn import datasets

# Importem el dataset
diagnostics = datasets.load_breast_cancer()
diagnostics_df = pd.DataFrame(diagnostics.data)

# Assignem una categoria a cada tumor

```

```

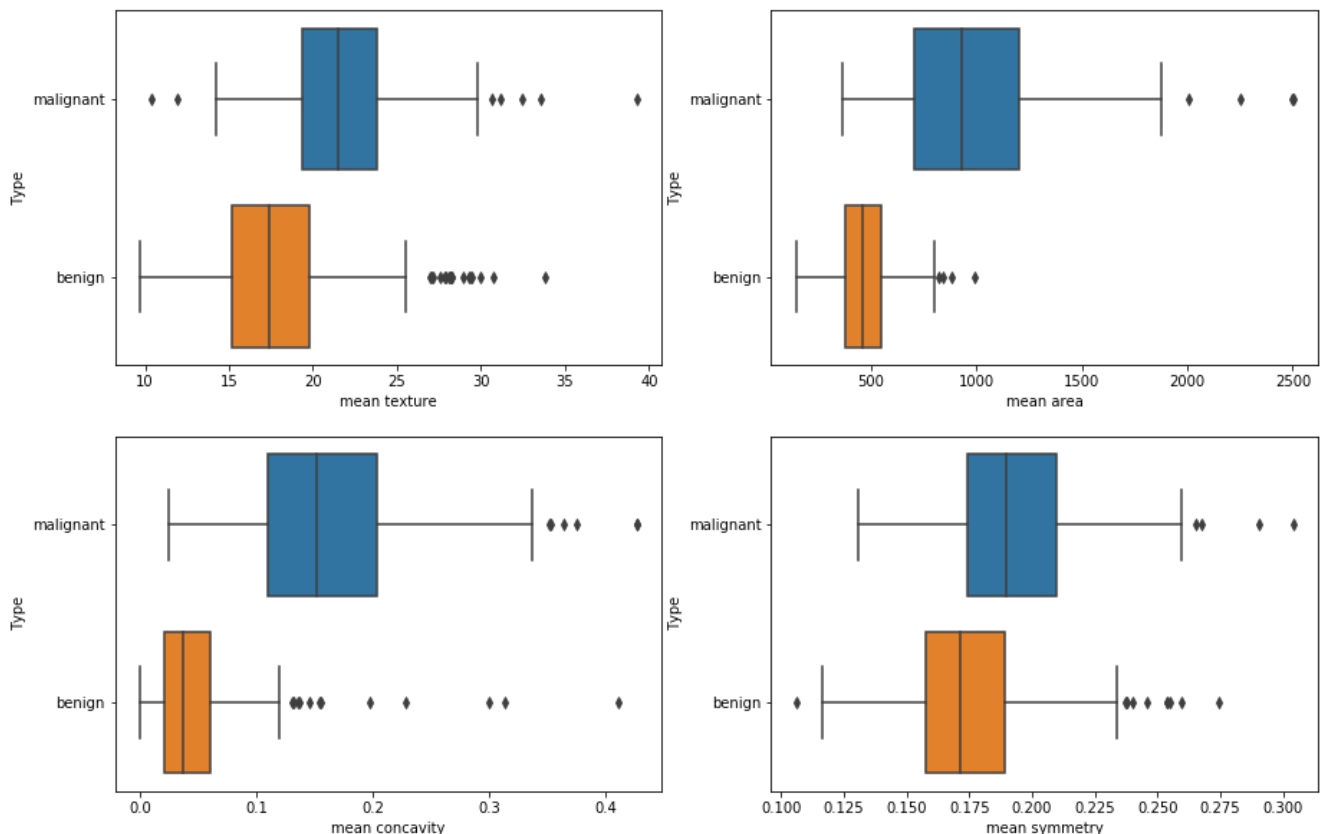
diagnostics_df.columns = diagnostics['feature_names']
diagnostics_df['Type'] = diagnostics.target
diagnostics_df.loc[(diagnostics_df.Type == 0), "Type"] = diagnostics.target_names[0]
diagnostics_df.loc[(diagnostics_df.Type == 1), "Type"] = diagnostics.target_names[1]

# Dibuixem cadascuna de les gràfiques
plt.figure(figsize=(15,10))
plt.subplot(2,2,1)
sns.boxplot(x=diagnostics_df['mean texture'],y=diagnostics_df.Type)
plt.subplot(2,2,2)
sns.boxplot(x=diagnostics_df['mean area'],y=diagnostics_df.Type)
plt.subplot(2,2,3)
sns.boxplot(x=diagnostics_df['mean concavity'],y=diagnostics_df.Type)
plt.subplot(2,2,4)
sns.boxplot(x=diagnostics_df['mean symmetry'],y=diagnostics_df.Type)

```

Out[22]:

<matplotlib.axes._subplots.AxesSubplot at 0x1a24c1ecd0>



Exercici 8

L'objectiu d'un model de **regressió lineal** és trobar una relació entre una o més característiques (variables independents) i una variable objectiu contínua (variable dependent). Quan només fem servir una característica predictiva se l'anomena **Regressió Lineal univariada** i si hi ha múltiples predictors es diu **Regressió Lineal Múltiple**.

Quan la variable dependent és una variable binària que conté dades codificades com 1 (sí, èxit, etc.) o 0 (no, fallada, etc.), com és el cas de l'dataset `breast_cancer`, haurem de fer servir una [regressió logística](#).

Crea un model de regressió logística que estimi la probabilitat que un tumor sigui maligne donada la seva àrea i la seva textura. Mostra la [matriu de confusió](#) del model obtingut.

Nota: Potser t'interessa explorar la funció [sklearn.metrics.confusion_matrix\(\)](#)

In [34]:

```

# Resposta
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

```

```

X = diagnostics_df[['mean texture', 'mean area']]
y = diagnostics_df['Type']

# Dividim les dades en dades d'entrenament (60%) i de test (40%) mitjançant la funció
train_test_split
train, test = train_test_split(diagnostics_df, test_size=0.4)
train_X = train[['mean texture', 'mean area']]
train_y = train['Type']
test_X = test[['mean texture', 'mean area']]
test_y = test['Type']

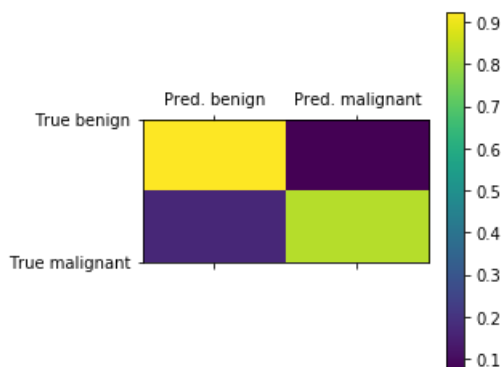
# Apliquem la regressió
clf = LogisticRegression(random_state=0, solver='lbfgs').fit(train_X, train_y)
clf.score(test_X, test_y)

# Predim el tipus de càncer atesos els predictors textura i Àrea
predicted = clf.predict(diagnostics_df[['mean texture', 'mean area']].values)
real = diagnostics_df['Type'].values

# Representem la matriu de confusió
import matplotlib.pyplot as plt
result_confusion_mat = []
for real_c in ['benign', 'malignant']:
    tmp = []
    for pred_c in ['benign', 'malignant']:
        # Dividim per el nom total de casos per calcular la proporció de:
        # Vertaders positius, vertaders negatius, falsos positius, i falsos negatius
        tmp.append( np.where(np.logical_and(predicted==pred_c, real==real_c))[0].size / np.where(real==real_c)[0].size )
    result_confusion_mat.append(tmp)

plt.matshow(result_confusion_mat)
plt.xticks( [0,1], ['Pred. benign', 'Pred. malignant'] )
plt.yticks( [0,1], ['True benign', 'True malignant'] )
plt.colorbar()
plt.show()

```



Exercici 9

El **Multi-layer Perceptron (MLP)** és un algorisme d'aprenentatge supervisat que aprèn una funció mitjançant l'entrenament d'un model que associa n dimensions d'entrada (*predictors*) amb dimensions de sortida (*targets*). Tenint en compte un conjunt de funcions i un objectiu, pot aprendre un aproximador de funcions no lineals per classificació o regressió. És diferent de la regressió logística, ja que entre la capa d'entrada i la de sortida, pot haver-hi una o més capes no lineals, anomenades capes ocultes (*hidden layers*).

Aplica un classificador basat en un MLP per predir el tipus de tumor utilitzant l'àrea, la textura, la simetria i la concavitat, com a atributs i utilitzant el 70% de les mostres d'entrenament i el 30% de test. Heu d'utilitzar la funció [sklearn.neural_network.MLPClassifier](#).

Utilitza tres nivells (*layers*) amb 30 neurones per nivell. Quin valor de precisió obtenim en un model basat en un MLP?

In []:

```

# Resposta
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# Dividim les dades en dades d'entrenament (70%) i de test (30%) mitjançant la funció

```

```
# Dividim les dades en dades d'entrenament (70%) i de test (30%) mitjançant la funció
train_test_split
train,test = train_test_split(diagnostics_df, test_size=0.3)
train_X = train[['mean texture','mean area','mean symmetry','mean concavity']]
train_y = train['Type']
test_X = test[['mean texture','mean area','mean symmetry','mean concavity']]
test_y = test['Type']

# Apliquem el MLP
clf = MLPClassifier(hidden_layer_sizes=(3, 30), max_iter=1000)
clf.fit(train_X, train_y)

# Mostrem la precisió
score = clf.score(test_X, test_y)
print("El model té un " + str(np.around(score,2)*100) + '% de precisió en predir el tipus de tumor.
' )
```

Exercici 10

Apliqueu un classificador basat en un [arbre de decisió](#) d'un màxim de 3 nivells de profunditat per predir els tipus de tumor utilitzant l'àrea, la textura i la simetria com a atributs i utilitzant 60% de les mostres d'entrenament i el 40% de test. Heu d'utilitzar la funció [sklearn.tree.DecisionTreeClassifier](#).

Quin valor de precisió obtenim en un model basat en un arbre de decisió? Representa l'arbre de decisió i exporta-ho a un arxiu PDF. Explora l'arbre resultant al PDF i respon a la següent pregunta: Donades aquestes dades, com diagnosticaries a un pacient que presenta un tumor d'àrea 400, simetria 0.18, i un valor de textura de 19 punts?

Nota: Potser la funció [tree](#) de *sklearn* sigui d'utilitat.

In []:

```
# Respuesta
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn import tree

# Dividim les dades en dades d'entrenament (60%) i de test (40%) mitjançant la funció
train_test_split
train,test = train_test_split(diagnostics_df, test_size=0.40)

# Creem el classificador
dtree = DecisionTreeClassifier(max_depth=3)

# Entrenem el classificador
fit = dtree.fit(train[['mean area','mean symmetry', 'mean texture']],train[['Type']])

# Avaluem la precisió del classificador
score = dtree.score(test[['mean area','mean symmetry', 'mean texture']],test[['Type']])
print("El model té un " + str(np.around(score,2)*100) + '% de precisió en predir el tipus de tumor.
' )

# Mostrem l'arbre de decisió i el exportem a un arxiu PDF
fig = plt.figure(figsize=(20,20), facecolor='w')
tree.plot_tree(fit, filled=True, feature_names=['mean area','mean symmetry', 'mean texture'],
class_names=['benigno','maligno']);
plt.savefig('tree.pdf')
```

Resposta

Classificarem el tumor com benigne (corresponent amb el primer full de l'arbre) amb una probabilitat d'error (impuresa de Gini) bastant baixa.